

UNIT 1

Basic Concepts

1. What is digital logic? Why is binary preferred over other number systems in digital computers?
2. Define a logic gate. Explain the importance of logic circuits in digital computers.
3. Differentiate between combinational and sequential logic circuits.

Logic Gates

4. Explain the working of basic logic gates: AND, OR, NOT, NAND, NOR, XOR, and XNOR with truth tables.
5. How are logic gates constructed using MOSFETs?
6. What are universal gates? How can NAND and NOR gates be used to implement other gates?

Boolean Algebra

7. Define Boolean algebra and list its basic identities.
8. State and prove De Morgan's Theorems with examples.
9. How can a Boolean function be represented using a truth table, Boolean expression, and logic diagram?

Logic Circuit Design

10. What are minterms and maxterms in Boolean algebra? Give examples.
11. Simplify the Boolean function $F = ABC + A'BC + AB'C'$ using Boolean identities.
12. Describe the concept of Complement of a Boolean Function with an example.

Karnaugh Maps (K-Maps)

13. What is a Karnaugh Map (K-Map)? Explain its significance in Boolean function simplification.
14. Explain how to simplify a Boolean function using a 2-variable K-map with an example.
15. Solve the following Boolean function using a 4-variable K-map:
 $F(A, B, C, D) = \Sigma(1, 3, 7, 11, 15)$

Combinational Logic Circuits

16. Design a Half Adder and Full Adder using logic gates and explain their truth tables.

17. What is a multiplexer? Explain a 4:1 and 8:1 multiplexer with a logic diagram.
18. Describe the working of a decoder and an encoder with block diagrams.
19. Explain the Parity Checker and Parity Generator with their applications.

Flip-Flops and Sequential Circuits

20. Explain the working of an SR Flip-Flop with a truth table and logic circuit diagram.
21. Differentiate between latch and flip-flop with an example.
22. What is a JK Flip-Flop? How does it eliminate the indeterminate state of an SR Flip-Flop?
23. What is an edge-triggered flip-flop? Explain its working with a circuit diagram.
24. How does a Master-Slave Flip-Flop work? Explain with a block diagram.
25. What are shift registers? Explain the working of a 4-bit shift register.
26. Explain the working of a counter with the help of a 3-bit ripple counter.

Memory Components and Integrated Circuits

27. What is RAM? Explain the organization of a RAM module with a block diagram.
28. What is ROM? Explain the differences between ROM, PROM, and EPROM.
29. Discuss the characteristics of CMOS circuits and their advantages over TTL circuits.
30. Explain the concepts of Fan-in and Fan-out and their effect on digital circuit design.

UNIT 2: MEMORY AND INSTRUCTION SEQUENCING

Memory Organization

30. Explain the concept of **memory locations and addresses**.
31. What is **byte addressability**? How does it differ from word addressing?
32. Differentiate between **Big-Endian and Little-Endian** memory assignments.
33. What is **word alignment**? Why is it important in memory organization?

Memory Operations

34. Describe the **Read and Write operations** in memory access.
35. What is the function of the **Program Counter (PC)** in instruction execution?

Instruction Types & Formats

36. Explain **three-address, two-address, and one-address** instruction formats with examples.
37. How does a **stack-based architecture** differ from a **register-based architecture**?
38. What is **Register Transfer Notation (RTN)**? Explain with examples.

RISC vs. CISC Architecture

39. What are the differences between **RISC and CISC** instruction sets?
40. Explain the advantages of **RISC architecture** over **CISC architecture**.

Instruction Execution & Sequencing

41. What is **straight-line sequencing** in instruction execution?
42. Explain the **fetch-decode-execute** cycle of a CPU.
43. How is a **branch instruction** used to implement loops in assembly language?

Addressing Modes

44. What are **addressing modes**? Why are they important in instruction execution?
45. Explain the following addressing modes with examples:
 - Immediate Addressing
 - Register Addressing
 - Direct (Absolute) Addressing
 - Indirect Addressing
46. What is **Indexed Addressing Mode**? How is it useful in array processing?
47. What is the **difference between relative and indexed addressing**?
48. How does **auto-increment and auto-decrement addressing mode** work?

Assembly Language & Assembler Directives

49. Explain the concept of **assembly language** and its advantages over machine language.
50. What are **assembler directives**? Explain with examples.
51. What is a **symbol table** in an assembler? Why is it needed?

BASIC INPUT/OUTPUT & PROCESSOR OPERATIONS

Input/Output Basics

1. Explain how **I/O devices communicate** with the processor and memory.
2. Differentiate between **program-controlled I/O** and **memory-mapped I/O**.
3. How does the processor handle **I/O synchronization** with devices of different speeds?
4. Explain the role of **status flags (SIN and SOUT)** in I/O operations.

Program-Controlled I/O

5. What is **program-controlled I/O**? Explain with an example.
6. Describe the **wait loop mechanism** used in program-controlled I/O.
7. How does a processor transfer data from the **keyboard to memory**?
8. Explain the **process of transferring data from the processor to the display**.
9. What is the role of a **device interface** in I/O operations?

Memory-Mapped I/O

10. What is **memory-mapped I/O**? How does it simplify device communication?
 11. Write the sequence of instructions to perform an **I/O read operation using memory-mapped I/O**.
 12. Write the sequence of instructions to perform an **I/O write operation using memory-mapped I/O**.
 13. Explain how **status registers (INSTATUS, OUTSTATUS)** are used in memory-mapped I/O.
-

STACKS & QUEUES

Stacks

14. What is a **stack**? How does it follow the **Last-In-First-Out (LIFO)** principle?
15. Explain how a **stack pointer (SP)** is used in stack operations.
16. What is the difference between a **push and pop operation** in a stack?
17. Implement the **push and pop operations** using **autoincrement and autodecrement addressing modes**.
18. How can a stack be used to detect **empty and full errors**?

Queues

19. What is a **queue**? How does it follow the **First-In-First-Out (FIFO)** principle?
 20. How are **front and rear pointers** used in queue operations?
 21. What is a **circular buffer**? How does it help in queue management?
-

UNIT 2B: SUBROUTINES & FUNCTION CALLS

6. Subroutines

22. What is a **subroutine**? Why are subroutines important in programming?
23. Explain the **subroutine linkage mechanism** using a **link register**.
24. What is a **Call instruction**? How does it work?
25. What is a **Return instruction**? How does it work?

7. Subroutine Nesting & Stack Frames

26. What is **subroutine nesting**? How does it affect return address storage?
27. Why is a **stack used for storing return addresses** during subroutine calls?
28. Explain how a **processor stack** is used in subroutine execution.
29. What is a **stack frame**? How does it help in subroutine execution?
30. What is a **frame pointer (FP)**? How does it assist in accessing local variables?

Parameter Passing in Subroutines

31. Explain **parameter passing by value** with an example.
 32. Explain **parameter passing by reference** with an example.
 33. How can a **stack be used for parameter passing** in subroutines?
-

MACHINE INSTRUCTIONS & ENCODING

Logical & Shift Instructions

34. Explain the **logical operations** AND, OR, NOT with examples.
35. What are **Shift instructions**? Differentiate between **Logical and Arithmetic shifts**.

- 36. How does an **Arithmetic Shift Right (AShiftR)** affect signed numbers?
- 37. What are **Rotate instructions**? How do they differ from shift instructions?

Multiplication & Division Instructions

- 38. How does **signed multiplication** work in assembly instructions?
- 39. Explain **division instruction** in assembly language.

Encoding of Machine Instructions

- 40. What is **machine instruction encoding**? Why is it necessary?
- 41. What is an **opcode**? How is it represented in a machine instruction?
- 42. How are **registers represented** in an encoded instruction?
- 43. Explain the encoding format of the instruction **Move 24(R0), R5**.
- 44. Explain how **immediate operands** are encoded in machine instructions.
- 45. Why do **branch instructions** use **relative addressing** instead of absolute addresses?

CISC vs. RISC Architecture

- 46. What is the difference between **CISC and RISC** architectures?
- 47. Explain the **advantages and disadvantages** of **CISC** architecture.
- 48. Explain the **advantages and disadvantages** of **RISC** architecture.
- 49. Why do **RISC processors** rely heavily on **registers** for computations?
- 50. How does **instruction pipelining** benefit **RISC architecture**?